Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 4

По дисциплине Виртуализация сетевых функций

Тема работы SDN: OpenFlow 1.3, Open vSwitch и контроллер OpenDaylight

Обучающийся Титов Георгий Константинович

Факультет Прикладной информатики

Группа К3422

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Титов Г. К.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Стр.

1 Цель работы и персональные параметры 3

2 Среда выполнения, топология и архитектура управления 4

3 Развёртывание контроллера и подключение OVS 6

3.1 Создание лаборатории и настройка сервисов 6

3.2 Подтверждение соединения и RESTCONF-инвентарь 6

4 Проактивная пересылка: до и после установки потоков 9

4.1 Проверка без пользовательских потоков 9

4.2 Установка восьми пересылающих правил 10

4.3 Контроль связности после установки 11

5 ACL для ICMP и статистика OpenFlow 13

5.1 Правило селективной блокировки 13

5.2 Сравнение счётчиков OVS и ODL 14

6 Отказ контроллера: secure, standalone и восстановление 17

6.1 Остановка ODL в режиме secure 17

6.2 Secure с пустыми таблицами 18

6.3 Переключение на standalone 18

6.4 Возврат ODL и восстановление правил 19

7 Ответы на контрольные вопросы 21

7.1 Как называется управление через отдельную management-сеть? 21

7.2 Зачем нужен table-miss-enforcer? 21

7.3 Почему до потоков не проходит ping? 21

7.4 Чем отличаются config и operational? Что после del-flows? .. 22

7.5 Что безопаснее и надёжнее? Что такое reconciliation? 22

8 Заключение 23

1 Цель работы и персональные параметры

Цель работы — построить SDN-сеть с OpenDaylight и двумя Open vSwitch, изучить RESTCONF-инвентарь, программирование таблиц OpenFlow 1.3, селективную фильтрацию ICMP и поведение плоскости данных при отказе контроллера.

Параметр	Значение
Студент (фамилия)	Титов
Полное ФИО	Титов Георгий Константинович
Группа	K3422
Номер ИСУ	409687
Вариант	STUDENT=titov, N=87
Приоритет ACL	587
Дата экспериментов	05.11.2026

Таблица 1.1 — Персональные параметры

2 Среда выполнения, топология и архитектура управления

Лаборатория выполнялась в Ubuntu 24.04 (WSL2) с Docker и Containerlab 0.82.0. Контроллер — OpenDaylight Chromium 0.24.0 на Karaf/Java 21. Коммутаторы — Open vSwitch в контейнерах, br0 работает с datapath_type=netdev; тестовые хосты — h1, h2 и h3.

Параметр	Фактическое значение
STUDENT / N	titov / 87
Префикс Flow ID	titov-
Management-подсеть	10.104.87.0/24
OpenDaylight / RESTCONF	10.104.87.100; TCP/6653 и TCP/8181
OVS s1 / DPID	10.104.87.11 / openflow:1
OVS s2 / DPID	10.104.87.12 / openflow:2
Хост h1	10.87.4.1/24
Хост h2	10.87.4.2/24
Хост h3	10.87.4.3/24
ACL	priority = 500 + 87 = 587

Таблица 2.1 — Параметры лаборатории

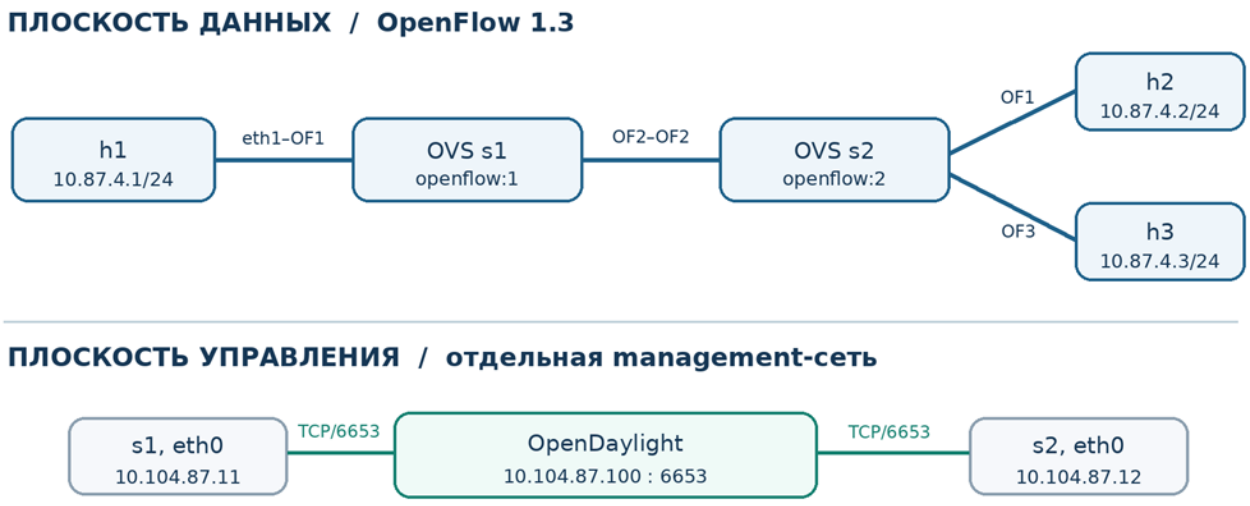


Рисунок 2.1 — Схема соединений: data plane с портами OpenFlow и отдельная management-сеть.

В плоскости данных h1 подключён к s1:OF1; линк s1:OF2 ↔ s2:OF2; h2 подключён к s2:OF1; h3 — к s2:OF3. Управляющий канал OVS ↔ ODL

проходит по eth0 через 10.104.87.0/24, а не по пользовательским портам eth1–eth3.

3 Развёртывание контроллера и подключение OVS

3.1 Создание лаборатории и настройка сервисов

В рабочем каталоге создан YAML-файл топологии с шестью узлами, двумя пользовательскими коммутаторами и тремя хостами. Containerlab создаёт контейнеры и виртуальные сетевые соединения. В ODL активируются функции OpenFlow Plugin, table-miss-enforcer и RESTCONF; затем скрипт ovs-setup.sh формирует br0 на каждом OVS, назначает datapath ID 1/2, номера портов и адрес контроллера.

```
cd ~/nfv-labs-titov/lab4
export STUDENT=titov N=87
containerlab deploy -t lab4.clab.yml
#   Karaf:
docker exec -t clab-titov-l4-odl /opt/odl/bin/client \
  'feature:install odl-openflowplugin-flow-services-rest odl-openflowplugin-app-ta
docker exec -i -e DPID=1 -e ODL_IP=10.104.87.100 \
  clab-titov-l4-s1 sh < scripts/ovs-setup.sh
docker exec -i -e DPID=2 -e ODL_IP=10.104.87.100 \
  clab-titov-l4-s2 sh < scripts/ovs-setup.sh
```

Листинг 3.1: Развёртывание и включение SDN-контроллера

Режим secure применяется изначально; datapath netdev позволяет работать без модуля OVS ядра хоста; ethtool tx off отключает разгрузку TX для корректной передачи TCP в userspace OVS.

3.2 Подтверждение соединения и RESTCONF-инвентарь

```
docker exec clab-titov-l4-s1 ovs-vsctl show
docker exec clab-titov-l4-s2 ovs-vsctl show
curl -fsS -u admin:admin -H 'Accept: application/yang-data+json' \
  'http://10.104.87.100:8181/rests/data/opendaylight-inventory:nodes?content=nonco
  | jq -r '."opendaylight-inventory:nodes".node[].id'
```

Листинг 3.2: Проверка обоих OVS и инвентаря OpenDaylight

По скриншотам 1–3 видно: у обоих мостов br0 контроллер tcp:10.104.87.100:6653 подключён (is_connected: true), fail_mode=secure; ответ RESTCONF содержит два DPID — openflow:1 и openflow:2.

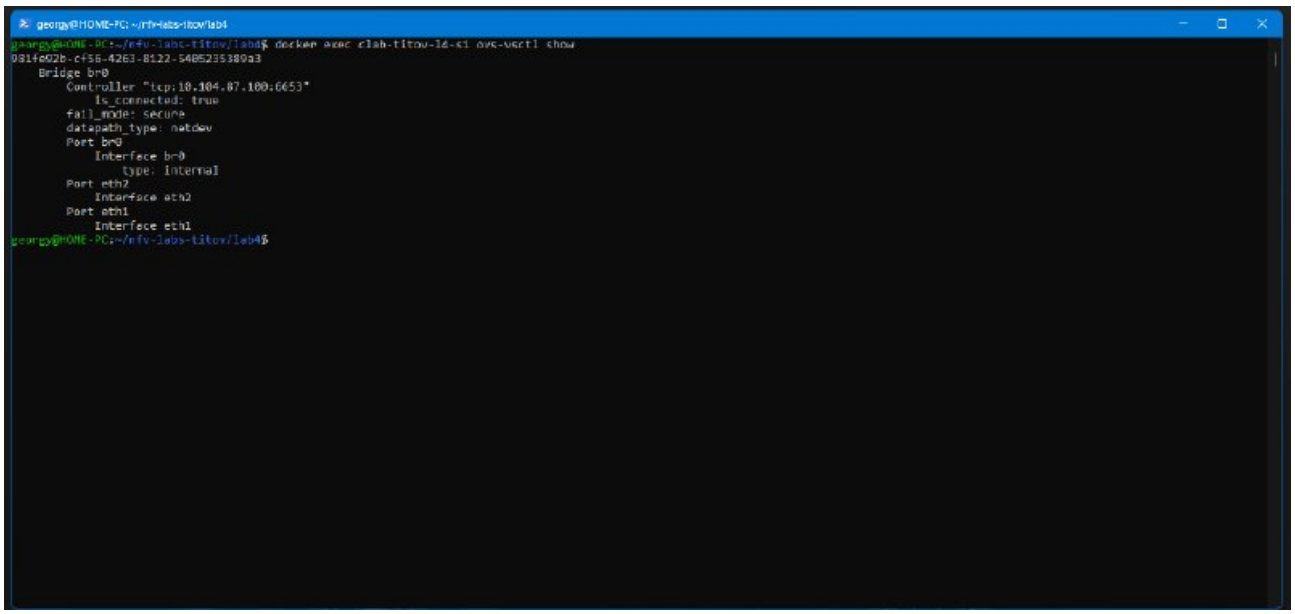


Рисунок 3.1 — Мост br0 коммутатора s1: адрес ODL и активное подключение.

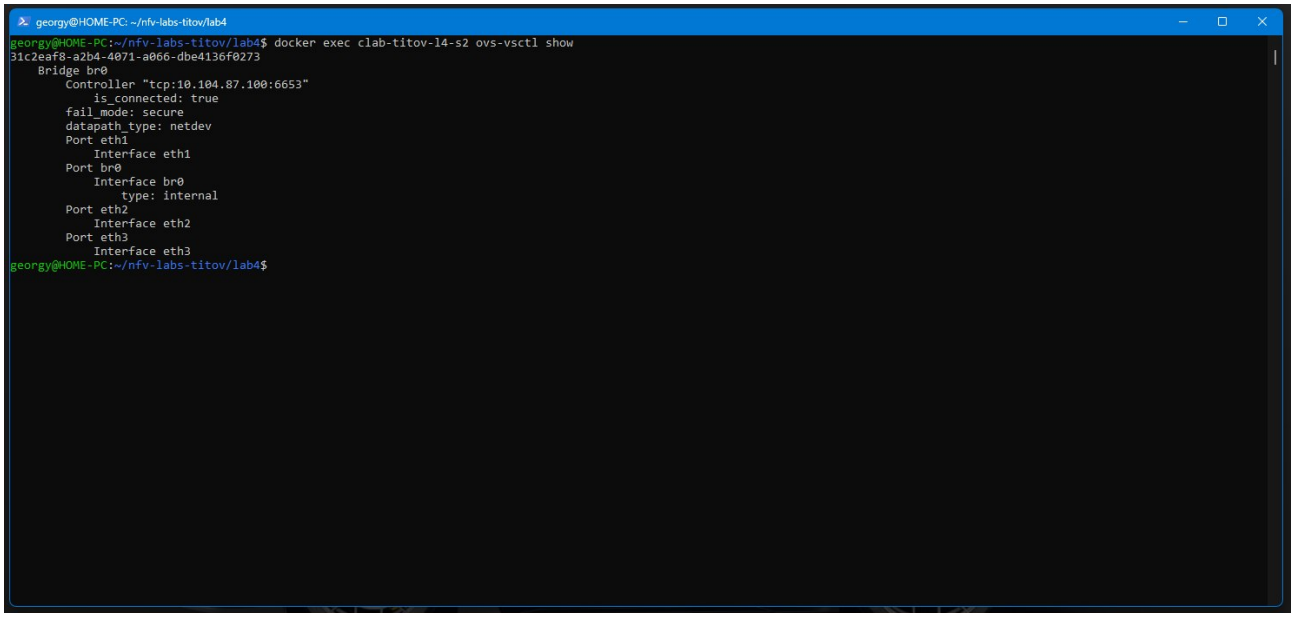


Рисунок 3.2 — Мост br0 коммутатора s2: активное подключение и порты eth1–eth3.

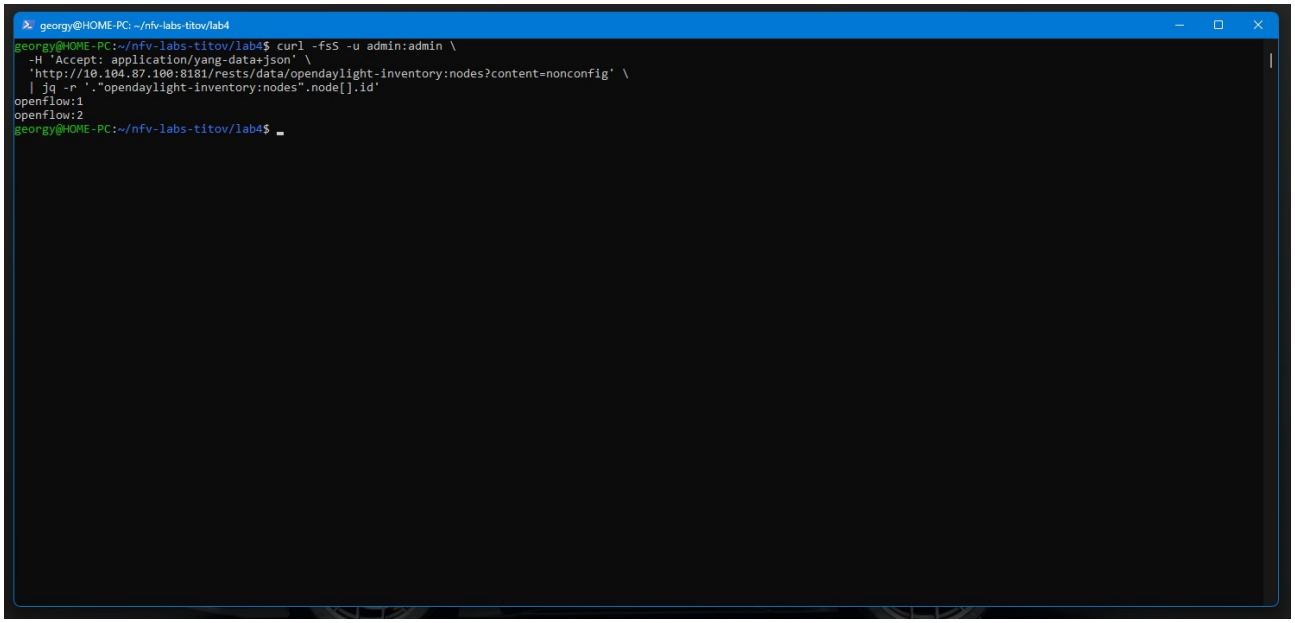


Рисунок 3.3 — Инвентарь RESTCONF: openflow:1 и openflow:2.

4 Проактивная пересылка: до и после установки потоков

4.1 Проверка без пользовательских потоков

Для демонстрации исходного состояния пересылающие правила были удалены через RESTCONF, после чего при активном fail_mode=secure проверена доступность h2 с h1. При отсутствии правил ARP/ICMP не получают действия output; ответный пакет не поступает на хост.

```
source scripts/odl-rest.sh
del_flow 1 titov-s1-1to2
del_flow 1 titov-s1-2to1
del_flow 2 titov-s2-1to2
del_flow 2 titov-s2-3to2
del_flow 2 titov-s2-to-h2-arp
del_flow 2 titov-s2-to-h2-ip
del_flow 2 titov-s2-to-h3-arp
del_flow 2 titov-s2-to-h3-ip
docker exec clab-titov-l4-h1 ping -c 3 -W 1 10.87.4.2
```

Листинг 4.1: Удаление правил и проверка ping до установки

Фактический результат: 3 пакета отправлено, 0 получено, 100% потерь.

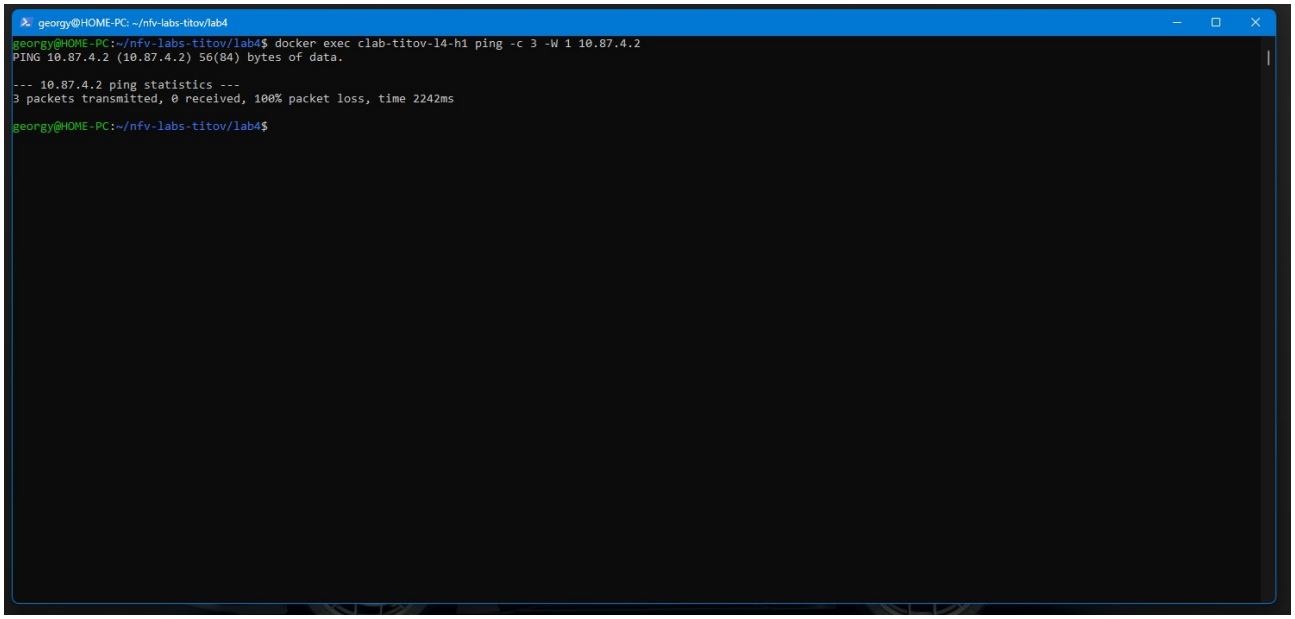


Рисунок 4.1 — Отсутствие связи h1 → h2 до установки пользовательских потоков.

4.2 Установка восьми пересылающих правил

На s1 настроены два потока по входному порту (OF1 → OF2 и OF2 → OF1). На s2 добавлены два потока для трафика от h2/h3 в сторону s1 и четыре адресных потока для IPv4/ARP от s1 к h2 и h3. Все восемь правил имеют priority=200 и идентификаторы с префиксом titov-.

Коммутатор	ID	Условие / действие
s1	titov-s1-1to2	in_port=1 → output:2
s1	titov-s1-2to1	in_port=2 → output:1
s2	titov-s2-1to2	in_port=1 → output:2
s2	titov-s2-3to2	in_port=3 → output:2
s2	titov-s2-to-h2-ip	in_port=2, IPv4 dst .2 → output:1
s2	titov-s2-to-h2-arp	in_port=2, ARP tpa .2 → output:1
s2	titov-s2-to-h3-ip	in_port=2, IPv4 dst .3 → output:3
s2	titov-s2-to-h3-arp	in_port=2, ARP tpa .3 → output:3

Таблица 4.1 — Установленные правила пересылки

```
./scripts/push-flows.sh --without-acl
docker exec clab-titov-l4-s1 ovs-ofctl -O OpenFlow13 dump-flows br0
docker exec clab-titov-l4-s2 ovs-ofctl -O OpenFlow13 dump-flows br0
```

Листинг 4.2: RESTCONF PUT и таблицы потоков

На скриншоте установки все восемь PUT получили HTTP 201; dump-flows подтверждает два правила на s1 и шесть на s2. Служебный table-miss имеет priority=0 и направляет неподходящий трафик контроллеру.

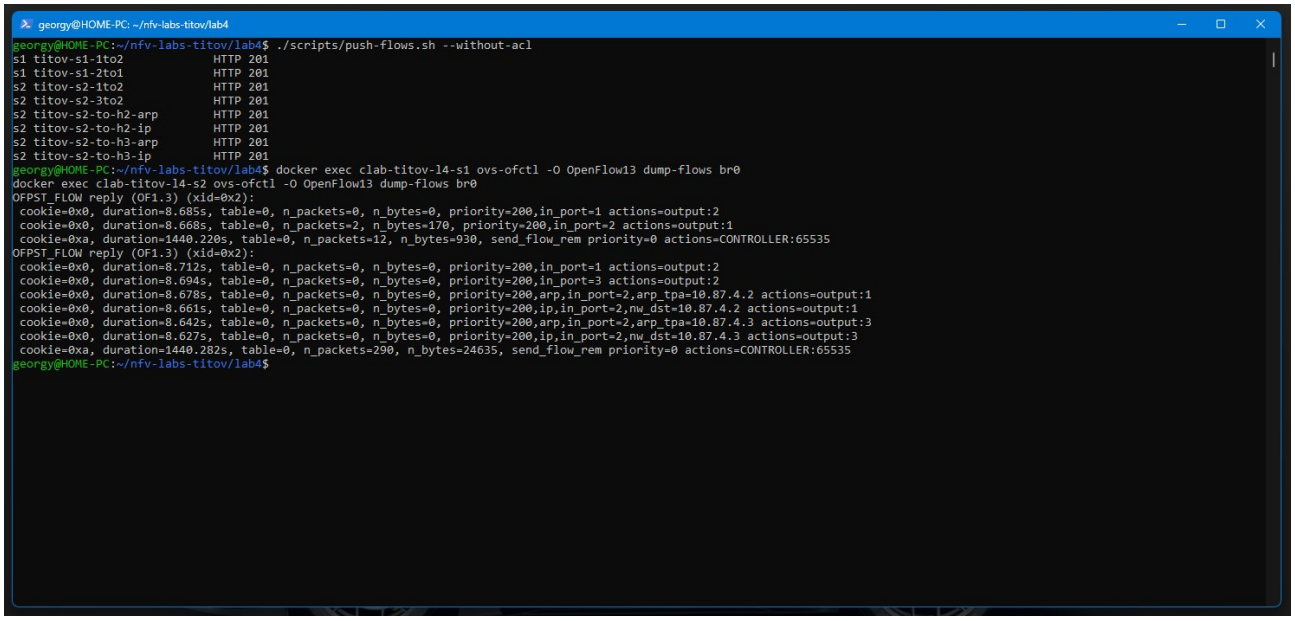


Рисунок 4.2 — Восемь успешных PUT и таблицы OpenFlow на s1 и s2.

4.3 Контроль связности после установки

```
docker exec clab-titov-l4-h1 ping -c 4 10.87.4.2
docker exec clab-titov-l4-h1 ping -c 4 10.87.4.3
```

Листинг 4.3: Проверка h1 → h2 и h1 → h3

Оба теста завершились без потерь: h1 → h2 — 4/4, среднее RTT 0,688 мс; h1 → h3 — 4/4, среднее RTT 0,740 мс (рисунок 7). Ненулевые счётчики потоков также ранее наблюдались в OVS, в частности на s2: IPv4 к h2 и h3 — по 4 пакета.

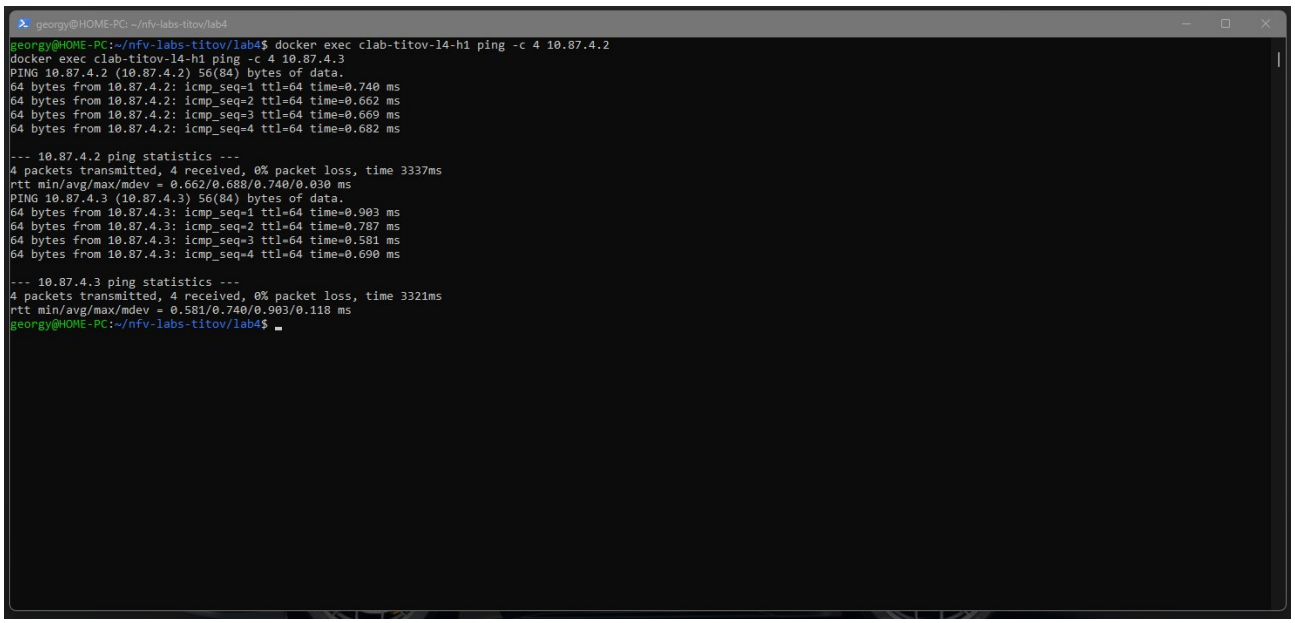


Рисунок 4.3 — Успешный ping к h2 и h3 после установки потоков.

5 ACL для ICMP и статистика OpenFlow

5.1 Правило селективной блокировки

Для варианта N=87 приоритет ACL равен 500+87=587, что выше обычных потоков (200). На s1, таблица 0, устанавливается поток titov-acl-icmp-h3: in_port=1, eth_type=IPv4, ip_proto=1 (ICMP), ipv4_dst=10.87.4.3/32; действие drop. HTTP/TCP не соответствует условию ip_proto=1, поэтому продолжает пересылаться.

```
docker exec -d clab-titov-l4-h3 python3 -m http.server 8080
source scripts/odl-rest.sh
put_flow 1 titov-acl-icmp-h3 flows/acl-icmp-h3.json
docker exec clab-titov-l4-s1 \
  ovs-ofctl -O OpenFlow13 dump-flows br0 | grep 'priority=587'
docker exec clab-titov-l4-h1 ping -c 5 -W 1 10.87.4.3
docker exec clab-titov-l4-h1 \
  curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
  http://10.87.4.3:8080/
```

Листинг 5.1: Установка ACL и два теста трафика

Результаты: PUT возвращает 201; OpenFlow показывает priority=587 и actions=drop; ping h1 → h3 даёт 5/5 потерь, а HTTP-запрос возвращает 200 OK.

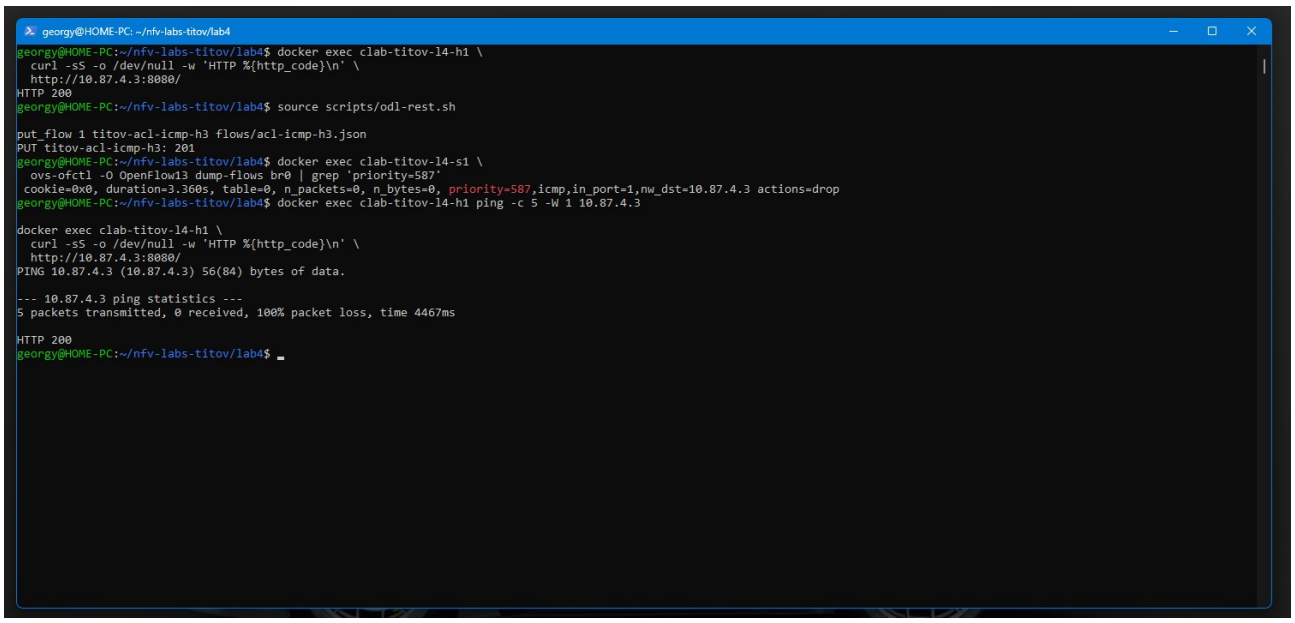


Рисунок 5.1 — ACL действует избирательно: ICMP блокируется, HTTP 200 проходит.

5.2 Сравнение счётчиков OVS и ODL

```
docker exec clab-titov-l4-s1 \
  ovs-ofctl -O OpenFlow13 dump-flows br0 | grep 'priority=587'
sleep 10
odl_get 'opendaylight-inventory:nodes/node=openflow%3A1/flow-node-inventory:table=
docker exec clab-titov-l4-h1 ping -c 3 -W 1 10.87.4.3
#  dump-flows  odl_get    .
```

Листинг 5.2: Чтение статистики до и после дополнительного ICMP

Первая серия: OVS n_packets=5, n_bytes=490, ODL packet-count=5, byte-count=490. Дополнительные три ICMP-пакета увеличили оба счётчика до 8 пакетов и 784 байт. Это доказывает обработку ACL в коммутаторе и получение статистики контроллером.

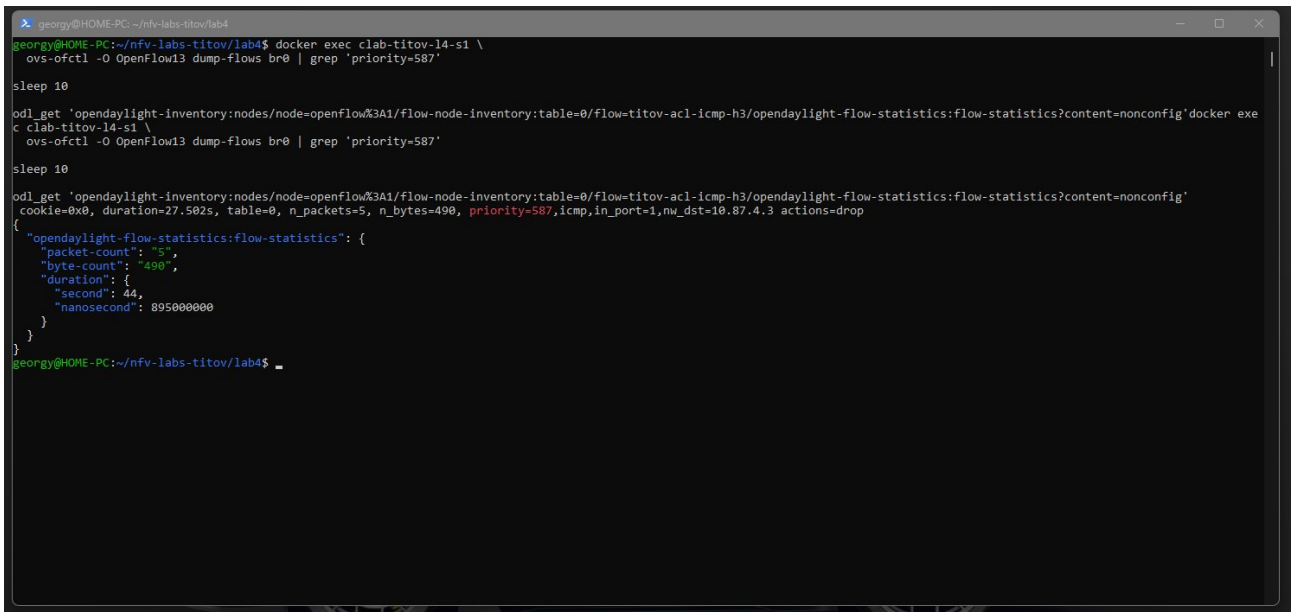


Рисунок 5.2 — Первые счётчики потока ACL: 5 пакетов и 490 байт в OVS и ODL.

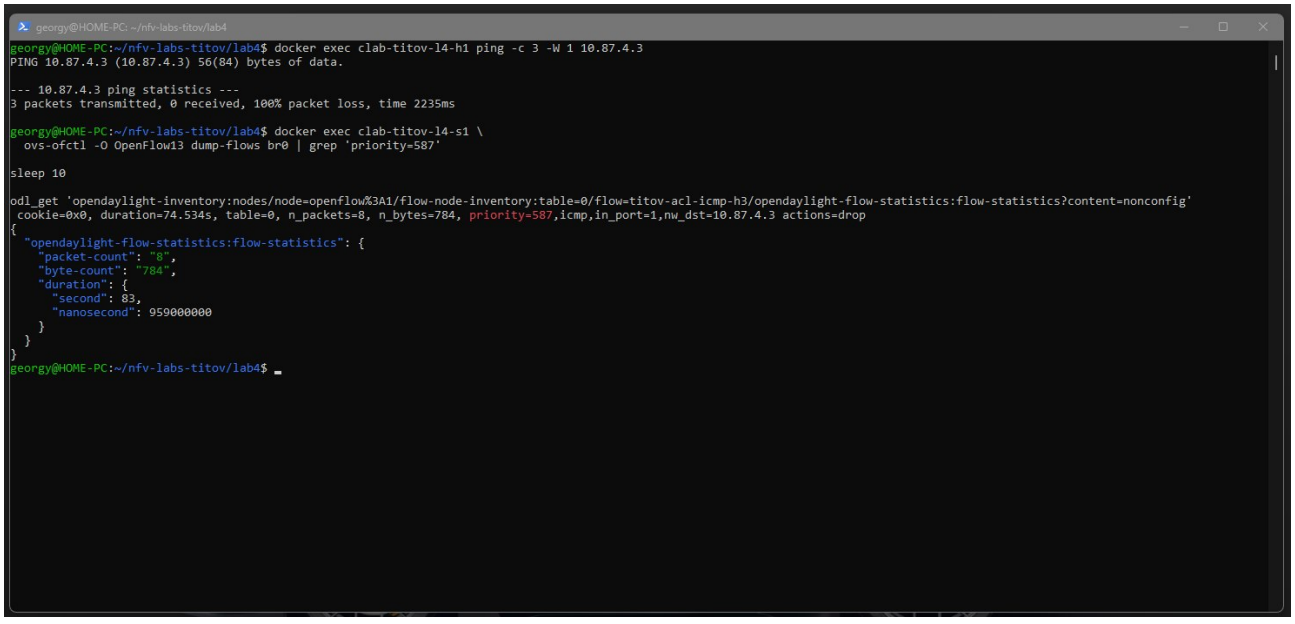


Рисунок 5.3 — Счётчики выросли после повторного ping: 8 пакетов и 784 байта.

По окончании опыта правило удалено RESTCONF DELETE (HTTP 204), а ICMP-связность h1 → h3 восстановилась; ранее наблюдалось 3/3 успешных ответов.

```
del_flow 1 titov-acl-icmp-h3
docker exec clab-titov-l4-h1 ping -c 3 10.87.4.3
```

Листинг 5.3: Удаление ACL

6 Отказ контроллера: secure, standalone и восстановление

6.1 Остановка ODL в режиме secure

На обоих коммутаторах установлен secure. После остановки ODL ранее установленные статические OpenFlow-правила продолжают пересылку. Поэтому связь сохраняется даже при исчезновении строки is_connected: true. В данном опыте ping h1 → h2 завершился успешно: 3/3, 0% потерь (рисунок 11).

```
docker exec clab-titov-l4-s1 ovs-vsctl get-fail-mode br0
docker exec clab-titov-l4-s2 ovs-vsctl get-fail-mode br0
docker stop clab-titov-l4-odl
sleep 20
docker exec clab-titov-l4-s1 ovs-vsctl show
docker exec clab-titov-l4-h1 ping -c 3 -W 1 10.87.4.2
```

Листинг 6.1: Потеря контроллера при сохранённых потоках

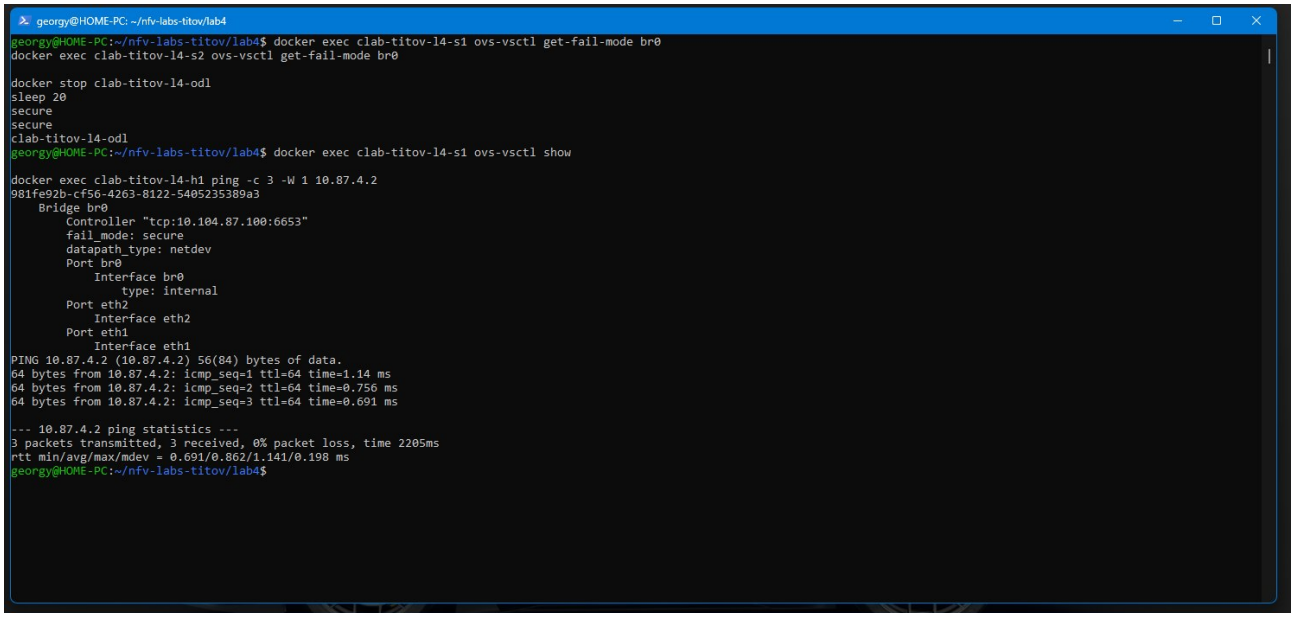


Рисунок 6.1 — ODL остановлен; secure сохраняет передачу по уже установленным правилам.

6.2 Secure с пустыми таблицами

После удаления потоков непосредственно на двух OVS новые пакеты не имеют подходящих действий пересылки. В отличие от standalone режим secure не создаёт собственную L2-политику. Ping h1 → h2: 3 отправлено, 0 получено, 100% потерь.

```
docker exec clab-titov-l4-s1 ovs-ofctl -O OpenFlow13 del-flows br0
docker exec clab-titov-l4-s2 ovs-ofctl -O OpenFlow13 del-flows br0
docker exec clab-titov-l4-s1 ovs-ofctl -O OpenFlow13 dump-flows br0
docker exec clab-titov-l4-h1 ping -c 3 -W 1 10.87.4.2
```

Листинг 6.2: Очистка таблиц и отсутствие связи в secure

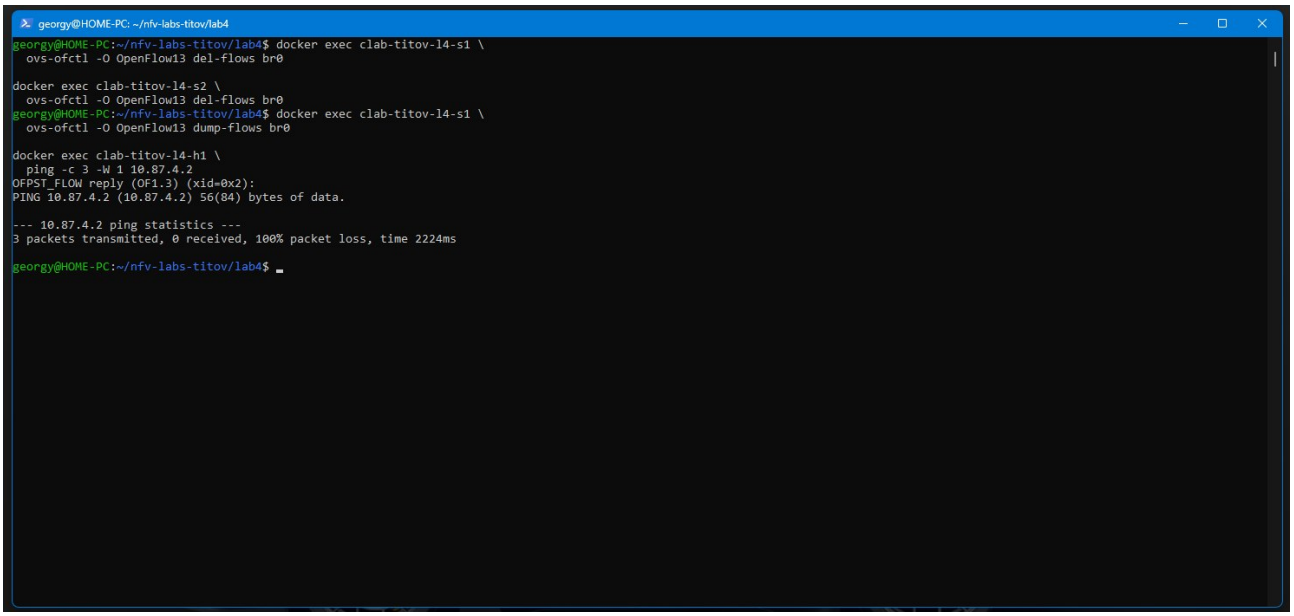


Рисунок 6.2 — В secure без потоков таблица пуста и ping не проходит.

6.3 Переключение на standalone

Режим standalone разрешает OVS использовать резервную локальную L2-коммутацию, когда контроллер недоступен. В этом тесте после переключения и ожидания 30 секунд обе пары восстановили связь: h2 → h3 — 4/4; h1 → h3 — 4/4. Вывод dump-flows мог оставаться пустым, так

как резервная коммутация не обязательно представлена пользовательскими потоками таблицы 0.

```
docker exec clab-titov-l4-s1 ovs-vsctl set-fail-mode br0 standalone
docker exec clab-titov-l4-s2 ovs-vsctl set-fail-mode br0 standalone
sleep 30
docker exec clab-titov-l4-s1 ovs-vsctl get-fail-mode br0
docker exec clab-titov-l4-s2 ovs-vsctl get-fail-mode br0
docker exec clab-titov-l4-h2 ping -c 4 -W 1 10.87.4.3
docker exec clab-titov-l4-h1 ping -c 4 -W 1 10.87.4.3
```

Листинг 6.3: Резервная L2-коммутация в standalone

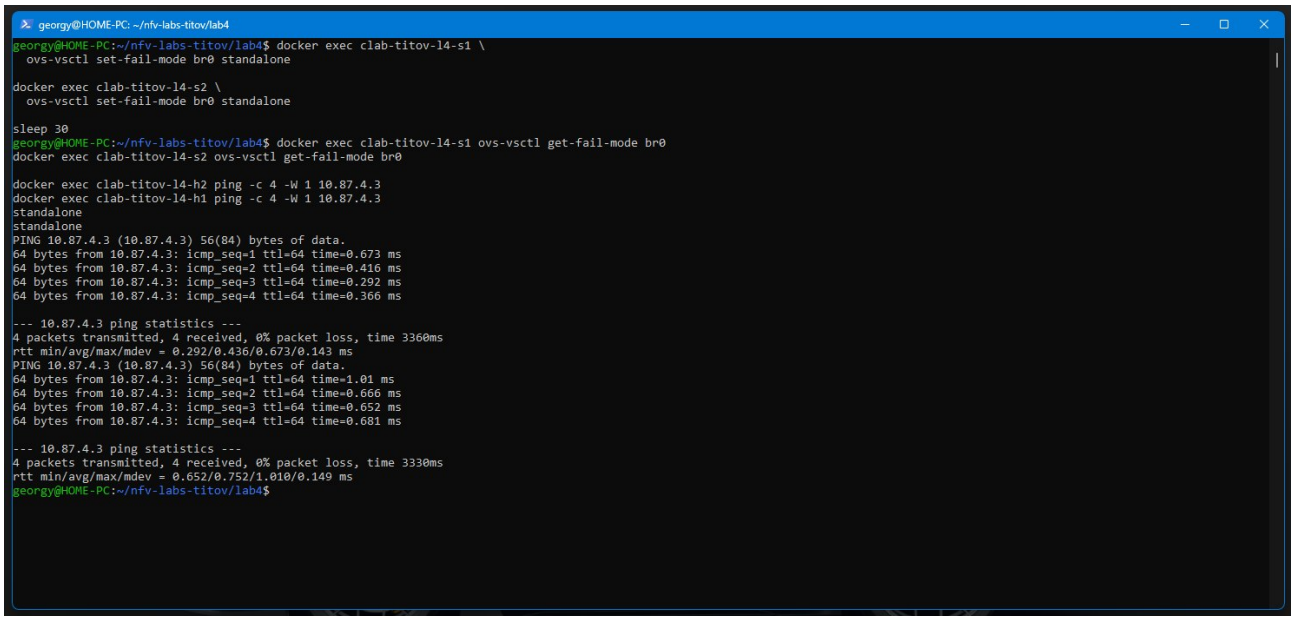


Рисунок 6.3 — В standalone с отключённым ODL оба ICMP-теста проходят.

6.4 Возврат ODL и восстановление правил

Оба OVS возвращены в режим secure; контейнер ODL запущен. После переподключения в ovs-vsctl show снова фиксировалось is_connected: true, RESTCONF возвращал openflow:1 и openflow:2. Восемь правил повторно применены скриптом push-flows.sh с ответами HTTP 204, итоговый ping h1 → h3 — 2/2, 0% потерь.

```
docker exec clab-titov-l4-s1 ovs-vsctl set-fail-mode br0 secure
```

```
docker exec clab-titov-l4-s2 ovs-vsctl set-fail-mode br0 secure
docker start clab-titov-l4-odl
sleep 90
docker exec clab-titov-l4-s1 ovs-vsctl show
docker exec clab-titov-l4-s2 ovs-vsctl show
./scripts/push-flows.sh --without-acl
./scripts/final-check.sh
```

Листинг 6.4: Восстановление контроллера, потоков и проверка

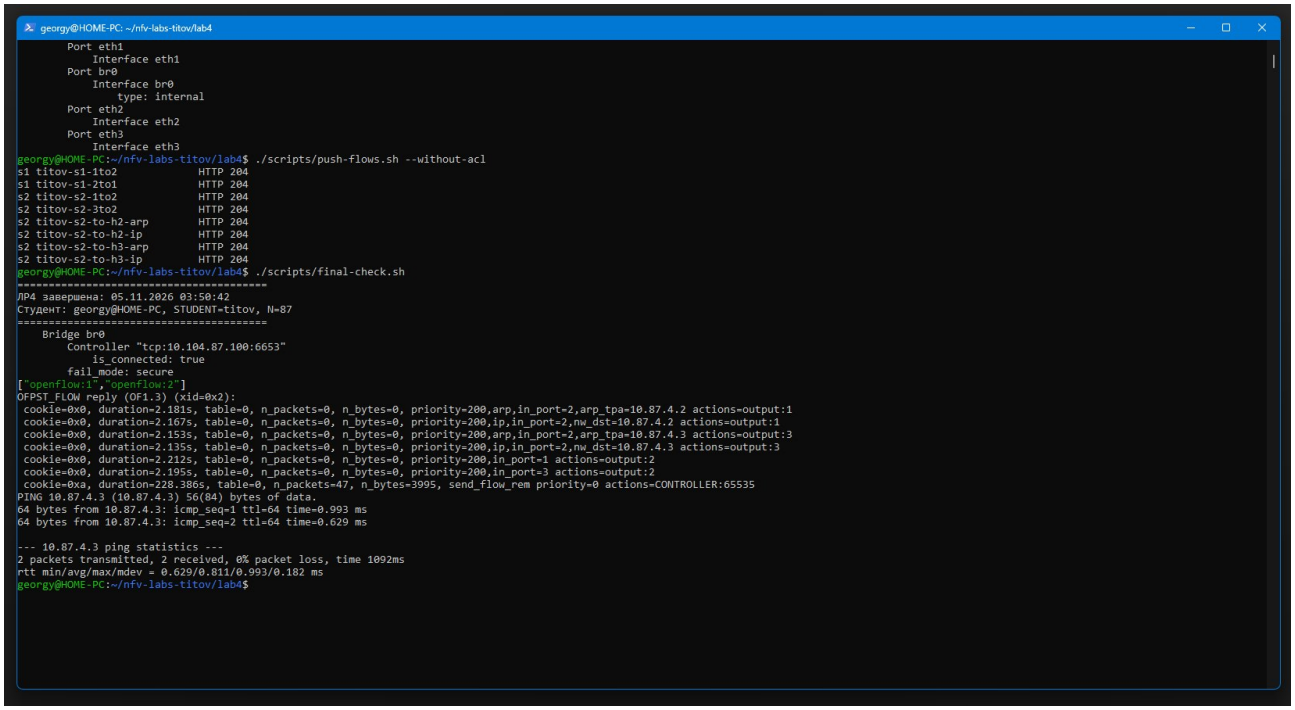


Рисунок 6.4 — Финальная проверка: ODL активен, два узла, восемь потоков и успешный ping.

Важно: повторный RESTCONF PUT выполнялся вручную; автоматическое восстановление удалённых потоков контроллером без повторного PUT отдельно не подтверждалось. В момент финального снимка счётчики новых пересылающих правил ещё равны нулю, поскольку снимок сделан непосредственно после установки; реальные ненулевые счётчики зафиксированы в предыдущих проверках.

7 Ответы на контрольные вопросы

7.1 Как называется управление через отдельную management-сеть?

Out-of-band management (внеполосное управление). Управляющий трафик между коммутаторами и ODL идёт через отдельную сеть 10.104.87.0/24 и интерфейсы eth0, а пользовательский трафик — через порты данных eth1–eth3. Это уменьшает зависимость доступности управления от состояния плоскости данных, облегчает изоляцию управляющих сервисов и ограничение доступа. При этом внеполосная сеть должна иметь собственную защиту и отказоустойчивость.

7.2 Зачем нужен table-miss-enforcer?

Приложение устанавливает служебный поток минимального приоритета (table-miss), чтобы обрабатывать пакеты, не совпавшие с более приоритетными правилами. В наблюдаемой конфигурации его действие — передача пакета контроллеру (CONTROLLER:65535). Это не означает автоматическую пересылку между портами: без приложения, определяющего дальнейшую обработку, ping до пользовательских потоков не проходит.

7.3 Почему до потоков не проходит ping?

Для ping требуется разрешить ARP (узнать MAC адрес назначения) и двунаправленную пересылку IPv4/ICMP. В secure на s1/s2 отсутствовали пользовательские match/action-потоки, направляющие кадры между портами. Пакеты без совпадения могли отправляться по table-miss контроллеру, но это не делает ODL адресатом ICMP и не заставляет его автоматически выполнить L2-пересылку. Поэтому ARP и ответы ping до хоста не доходили.

7.4 Чем отличаются config и operational? Что после del-flows?

Config datastore содержит желаемое состояние: созданные через RESTCONF записи потоков и их параметры (ID, match, priority, actions). Operational — оперативное наблюдаемое состояние: обнаруженные узлы, фактически установленные потоки, порты и статистика пакетов/байтов. Если ovs-ofctl del-flows удалить правило непосредственно на OVS, оно исчезнет из аппаратно/программно выполняемой таблицы, однако запись в конфигурации ODL может сохраниться. Контроллер может восстановить поток при согласовании состояния (reconciliation), но этот результат зависит от приложений и режима синхронизации. В лаборатории возвращение восьми правил после отказа подтверждено явными RESTCONF PUT.

7.5 Что безопаснее и надёжнее? Что такое reconciliation?

Secure безопаснее относительно выполнения централизованной SDN-политики: коммутатор не переключается по собственной инициативе в обычный learning-switch при отсутствии контроллера. Однако он может продолжать работать по уже установленным правилам; после очистки таблиц связь пропадает. Standalone повышает доступность базовой L2-связности в случае отказа ODL, но при этом может нарушать централизованно заданные ACL и правила сегментации. Reconciliation — процесс сверки желаемой конфигурации контроллера с фактическим состоянием коммутаторов и повторной установки расхождений после переподключения. Наш опыт подтвердил восстановление связности после ручной загрузки восьми правил; автоматическое reconciliation без PUT отдельно не проверялось.

8 Заключение

В ходе работы создана SDN-топология, плоскость управления отделена от пользовательской сети, программно установлены правила OpenFlow 1.3 и реализована выборочная блокировка ICMP. Результаты замеров подтвердили корректность ACL и различие fail_mode=secure/standalone при отказе контроллера. После восстановительных действий лаборатория возвращена в рабочее состояние без активного ACL.